

AI Assisted Coding Assignment-12.5

2303A51886 : 30 SAI SPURTHI. B 13.03.2026

Lab 12: Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

Lab Objectives:

- Apply AI-assisted programming to implement and optimize Week6 - Friday sorting and searching algorithms.
- Compare different algorithms in terms of efficiency and use cases.
- Understand how AI tools can suggest optimized code and complexity improvements.

Task Description #1 (Sorting – Merge Sort Implementation)

- Task: Use AI to generate a Python program that implements the Merge Sort algorithm.
- Instructions:
 - o Prompt AI to create a function `merge_sort(arr)` that sorts a list in ascending order.
 - o Ask AI to include time complexity and space complexity in the function docstring.
 - o Verify the generated code with test cases.
- Expected Output:
 - o A functional Python script implementing Merge Sort with proper documentation.

Release Notes: 1.111.0Lab-ASS-11.5.pyLab-ASS-12.5.py Xdata1.txtdata2.txtlibrary.py

Lab-ASS-12.5.py > ...
1 #Task-1 (Sorting - Merge Sort Implementation)
2 #Prompt:Generate a Python program that create a function merge_sort(arr) that sorts a list in ascending order
3 def merge_sort(arr):
4 """
5 Sorts a list in ascending order using the merge sort algorithm.
6 Time Complexity: O(n log n)
7 Space Complexity: O(n)
8 Args:
9 | arr (list): List of elements to be sorted.
10 Returns:
11 | list: Sorted list in ascending order.
12 """
13 if len(arr) <= 1:
14 return arr
15 mid = len(arr) // 2
16 left = merge_sort(arr[:mid])
17 right = merge_sort(arr[mid:])
18 return merge(left, right)
19 def merge(left, right):
20 result = []
21 i = j = 0
22 while i < len(left) and j < len(right):
23 if left[i] <= right[j]:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTSPython + - Python 3.8 (64-bit)

PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-ASS-12.5.py"
Test case 7: Input: [1, 2, 3, 4, 5] -> Sorted: [1, 2, 3, 4, 5]
Test case 8: Input: [5, 4, 3, 2, 1] -> Sorted: [1, 2, 3, 4, 5]
Test case 1: Input: [], Target: 1 -> Index: -1
Test case 2: Input: [1], Target: 1 -> Index: 0
Test case 3: Input: [1, 2, 3, 4, 5], Target: 3 -> Index: 2
Test case 4: Input: [1, 2, 3, 4, 5], Target: 6 -> Index: -1
Test case 5: Input: [10, 20, 30, 40, 50], Target: 30 -> Index: 2
Test case 6: Input: [10, 20, 30, 40, 50], Target: -10 -> Index: -1
Test case 7: Input: [1, 2, 3, 4, 5], Target: 1 -> Index: 0
Test case 8: Input: [1, 2, 3, 4, 5], Target: 5 -> Index: 4

Ln 341, Col 1 Spaces: 4 UTF-8 CRLF Python Python 3.8 (64-bit)

```
Release Notes: 1.111.0  Lab-ASS-11.5.py  Lab-ASS-12.5.py X  data1.txt  data2.txt  library.py
Lab-ASS-12.5.py > ...
19 def merge(left, right):
26     else:
27         result.append(right[j])
28         j += 1
29     result.extend(left[i:])
30     result.extend(right[j:])
31     return result
32 # Test cases
33 if __name__ == "__main__":
34     test_cases = [
35         [],
36         [1],
37         [5, 2, 9, 1, 5, 6],
38         [3, 2, 1],
39         [10, -1, 2, 5, 0],
40         [7, 7, 7, 7],
41         [1, 2, 3, 4, 5],
42         [5, 4, 3, 2, 1]
43     ]
44     for idx, case in enumerate(test_cases):
45         sorted_case = merge_sort(case)
46         print(f"Test case {idx + 1}: Input: {case} -> Sorted: {sorted_case}")
```

Task Description #2 (Searching – Binary Search with AI Optimization)

- Task: Use AI to create a binary search function that finds a target element in a sorted list.

- Instructions:

- o Prompt AI to create a function `binary_search(arr, target)` returning the index of the target or -1 if not found.

- o Include docstrings explaining best, average, and worst-case complexities.

- o Test with various inputs.

- Expected Output:

- o Python code implementing binary search with AI-generated comments and docstrings.

```
Release Notes: 1.111.0  Lab-ASS-11.5.py  Lab-ASS-12.5.py X  data1.txt  data2.txt  library.py
Lab-ASS-12.5.py > ...
50 #Task-2 (Searching - Binary Search with AI Optimization)
51 #Prompt:Create a binary search function binary_search(arr,target) returning the index of the target or -1
52 def binary_search(arr, target):
53     """
54     Performs binary search on a sorted list to find the index of the target element.
55
56     Time Complexity:
57         Best Case: O(1) - when the target is at the middle of the list.
58         Average Case: O(log n) - when the target is found after several iterations.
59         Worst Case: O(log n) - when the target is not found or is at the end of the list.
60
61     Space Complexity: O(1) - iterative implementation uses constant space.
62     Args:
63         arr (list): A sorted list of elements.
64         target: The element to search for.
65     Returns:
66         int: The index of the target element if found, otherwise -1.
67     """
68     left, right = 0, len(arr) - 1
69     while left <= right:
70         mid = left + (right - left) // 2
71         if arr[mid] == target:
72             return mid
```

```
Lab-ASS-12.5.py > ...
52 def binary_search(arr, target):
72     return mid
73     elif arr[mid] < target:
74         left = mid + 1
75     else:
76         right = mid - 1
77     return -1
78 # Test cases
79 if __name__ == "__main__":
80     test_cases = [
81         ([], 1),
82         ([1], 1),
83         ([1, 2, 3, 4, 5], 3),
84         ([1, 2, 3, 4, 5], 6),
85         ([10, 20, 30, 40, 50], 30),
86         ([10, 20, 30, 40, 50], -10),
87         ([1, 2, 3, 4, 5], 1),
88         ([1, 2, 3, 4, 5], 5)
89     ]
90     for idx, (arr, target) in enumerate(test_cases):
91         result = binary_search(arr, target)
92         print(f"Test case {idx + 1}: Input: {arr}, Target: {target} -> Index: {result}")
93
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Python + v [] [X]
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-ASS-12.5.py"
Test case 2: Input: [1], Target: 1 -> Index: 0
Test case 3: Input: [1, 2, 3, 4, 5], Target: 3 -> Index: 2
Test case 4: Input: [1, 2, 3, 4, 5], Target: 6 -> Index: -1
Test case 5: Input: [10, 20, 30, 40, 50], Target: 30 -> Index: 2
Test case 6: Input: [10, 20, 30, 40, 50], Target: -10 -> Index: -1
Test case 7: Input: [1, 2, 3, 4, 5], Target: 1 -> Index: 0
Test case 8: Input: [1, 2, 3, 4, 5], Target: 5 -> Index: 4
```

Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.
2. Sort appointments based on time or consultation fee.

Student Task

- Use AI to recommend suitable searching and sorting algorithms.
- Justify the selected algorithms.
- Implement the algorithms in Python.

```
#Task-3: Smart Healthcare Appointment Scheduling System
#Prompt:A healthcare platform maintains appointment records containing appointment ID, patient name, doctor
#Search appointments using appointment ID.
#Sort appointments based on time or consultation fee.
#Recommend suitable searching and sorting algorithms then justify the selected algorithms.Implement the alg
class Appointment:
    def __init__(self, appointment_id, patient_name, doctor_name, appointment_time, consultation_fee):
        self.appointment_id = appointment_id
        self.patient_name = patient_name
        self.doctor_name = doctor_name
        self.appointment_time = appointment_time
        self.consultation_fee = consultation_fee
class AppointmentSystem:
    def __init__(self):
        self.appointments = []
    def add_appointment(self, appointment):
        self.appointments.append(appointment)
    def search_appointment_by_id(self, appointment_id):
        for appointment in self.appointments:
            if appointment.appointment_id == appointment_id:
                return appointment
        return None
    # Merge Sort Algorithm to sort appointments by a given key (time or fee)
    def merge_sort(self, appointments, key):
        if len(appointments) > 1:
            mid = len(appointments) // 2
            left_half = appointments[:mid]
            right_half = appointments[mid:]
            # Recursive call on each half
            self.merge_sort(left_half, key)
```

```
Lab-ASS-12.5.py > ...
152 if __name__ == "__main__":
153     system = AppointmentSystem()
154     system.add_appointment(Appointment(1, "Alice", "Dr. Smith", "2024-07-01 10:00", 100))
155     system.add_appointment(Appointment(2, "Bob", "Dr. Jones", "2024-07-01 11:00", 150))
156     system.add_appointment(Appointment(3, "Charlie", "Dr. Brown", "2024-07-01 09:00", 120))
157     # Search for an appointment
158     appointment = system.search_appointment_by_id(2)
159     if appointment:
160         print(f"Found appointment: {appointment.patient_name} with {appointment.doctor_name} at {appointment.appointment_time} for {appointment.consultation_fee}")
161     else:
162         print("Appointment not found.")
163     # Sort appointments by time
164     system.sort_appointments_by_time()
165     print("Appointments sorted by time:")
166     for appointment in system.appointments:
167         print(f"{appointment.patient_name} with {appointment.doctor_name} at {appointment.appointment_time} for {appointment.consultation_fee}")
168     # Sort appointments by consultation fee
169     system.sort_appointments_by_fee()
170     print("Appointments sorted by consultation fee:")
171     for appointment in system.appointments:
172         print(f"{appointment.patient_name} with {appointment.doctor_name} at {appointment.appointment_time} for {appointment.consultation_fee}")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [ ] [ ]

PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-ASS-12.5.py"
Test case 8: Input: [1, 2, 3, 4, 5], Target: 5 -> Index: 4
Found appointment: Bob with Dr. Jones at 2024-07-01 11:00 for $150
Appointments sorted by time:
Charlie with Dr. Brown at 2024-07-01 09:00 for $120
Alice with Dr. Smith at 2024-07-01 10:00 for $100
Bob with Dr. Jones at 2024-07-01 11:00 for $150
Appointments sorted by consultation fee:
Alice with Dr. Smith at 2024-07-01 10:00 for $100
Charlie with Dr. Brown at 2024-07-01 09:00 for $120
Bob with Dr. Jones at 2024-07-01 11:00 for $150
```

Task Description #4: Railway Ticket Reservation System Scenario

A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.
2. Sort bookings based on travel date or seat number.

Student Task

- Identify efficient algorithms using AI assistance.
- Justify the algorithm choices.
- Implement searching and sorting in Python.

Lab-ASS-12.5.py > ...

```
177 #Task-4: Railway Ticket Reservation System
178 #Prompt:A railway reservation system stores booking details such as ticket ID, passenger name, train number
179 #Search tickets using ticket ID.
180 #Sort bookings based on travel date or seat number.
181 #Identify efficient algorithms.Justify the algorithm choices.Implement searching and sorting in Python.
182 from asyncio import Task
183 from datetime import datetime
184 class Ticket:
185     def __init__(self, ticket_id, passenger_name, train_number, seat_number, travel_date):
186         self.ticket_id = ticket_id
187         self.passenger_name = passenger_name
188         self.train_number = train_number
189         self.seat_number = seat_number
190         self.travel_date = travel_date
191 class ReservationSystem:
192     def __init__(self):
193         self.tickets = []
194     def add_ticket(self, ticket):
195         self.tickets.append(ticket)
196     # Search tickets by ticket ID
197     def search_ticket_by_id(self, ticket_id):
198         for ticket in self.tickets:
199             if ticket.ticket_id == ticket_id:
200                 return ticket
201         return None
202     # Merge Sort algorithm to sort tickets by travel date or seat number
203     def merge_sort(self, tickets, key):
204         if len(tickets) > 1:
205             mid = len(tickets) // 2
206             left_half = tickets[:mid]
207             right_half = tickets[mid:]
208             # Recursively split the list
```

```
Lab-ASS-12.5.py > ...
191 class ReservationSystem:
192     def __init__(self):
193         self.tickets = []
194         self.merge_sort(self.tickets, 'seat_number')
234
235 # Test cases
236 if __name__ == "__main__":
237     system = ReservationSystem()
238     # Adding sample tickets
239     system.add_ticket(Ticket(101, "Alice", "T1001", "A1", datetime(2024, 7, 1)))
240     system.add_ticket(Ticket(102, "Bob", "T1002", "B2", datetime(2024, 7, 3)))
241     system.add_ticket(Ticket(103, "Charlie", "T1003", "C3", datetime(2024, 7, 2)))
242     # Search for a ticket by ID
243     ticket = system.search_ticket_by_id(102)
244     if ticket:
245         print(f"Found ticket: {ticket.passenger_name} on train {ticket.train_number} in seat {ticket.seat_number}")
246     else:
247         print("Ticket not found.")
248     # Sort tickets by travel date
249     system.sort_tickets_by_date()
250     print("\nTickets sorted by travel date:")
251     for ticket in system.tickets:
252         print(f"{ticket.passenger_name} on train {ticket.train_number} in seat {ticket.seat_number} for {ticket.travel_date}")
253     # Sort tickets by seat number
254     system.sort_tickets_by_seat()
255     print("\nTickets sorted by seat number:")
256     for ticket in system.tickets:
257         print(f"{ticket.passenger_name} on train {ticket.train_number} in seat {ticket.seat_number} for {ticket.travel_date}")
258
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [ ] [X]
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-ASS-12.5.py"
Appointments sorted by consultation fee:
Alice with Dr. Smith at 2024-07-01 10:00 for $100
Charlie with Dr. Brown at 2024-07-01 09:00 for $120
Bob with Dr. Jones at 2024-07-01 11:00 for $150
```

Task Description #5: Smart Hostel Room Allocation System

A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.
2. Sort records based on room number or allocation date.

Student Task

- Use AI to suggest optimized algorithms.
- Justify the selections.
- Implement the solution in Python.

Lab-ASS-12.5.py > ...

```
262 #Task-5: Smart Hostel Room Allocation System
263 #Prompt:A hostel management system stores student room allocation details including student ID, room number
264 #Search allocation details using student ID.
265 #Sort records based on room number or allocation date.
266 #Suggest optimized algorithms.Justify the selections.Implement the solution in Python.
267 class RoomAllocation:
268     def __init__(self, student_id, room_number, floor, allocation_date):
269         self.student_id = student_id
270         self.room_number = room_number
271         self.floor = floor
272         self.allocation_date = allocation_date
273 class HostelSystem:
274     def __init__(self):
275         self.allocations = {}
276         # Add room allocation record (Using student ID as key)
277     def add_allocation(self, allocation):
278         self.allocations[allocation.student_id] = allocation
279         # Search allocation details by student ID (Optimized search using Hashing)
280     def search_by_student_id(self, student_id):
281         return self.allocations.get(student_id, None)
282         # Merge Sort for sorting by room number or allocation date
283     def merge_sort(self, allocations, key):
284         if len(allocations) > 1:
285             mid = len(allocations) // 2
286             left_half = allocations[:mid]
287             right_half = allocations[mid:]
288             # Recursively split the list
289             self.merge_sort(left_half, key)
290             self.merge_sort(right_half, key)
291             i = j = k = 0
292             # Merging the lists back together
```

```

Lab-ASS-12.5.py > ...
273 class HostelSystem:
283     def merge_sort(self, allocations, key):
292         # Merging the lists back together
293         while i < len(left_half) and j < len(right_half):
294             if getattr(left_half[i], key) < getattr(right_half[j], key):
295                 allocations[k] = left_half[i]
296                 i += 1
297             else:
298                 allocations[k] = right_half[j]
299                 j += 1
300             k += 1
301         while i < len(left_half):
302             allocations[k] = left_half[i]
303             i += 1
304             k += 1
305         while j < len(right_half):
306             allocations[k] = right_half[j]
307             j += 1
308             k += 1
309         # Sort records by room number
310         def sort_by_room_number(self):
311             self.merge_sort(list(self.allocations.values()), 'room_number')
312         # Sort records by allocation date
313         def sort_by_allocation_date(self):
314             self.merge_sort(list(self.allocations.values()), 'allocation_date')
315         # Test cases
316         if __name__ == "__main__":
317             system = HostelSystem()
318             # Adding allocation records
319             system.add_allocation(RoomAllocation(101, 202, 2, datetime(2024, 7, 1)))
320             system.add_allocation(RoomAllocation(102, 101, 1, datetime(2024, 7, 3)))
321             system.add_allocation(RoomAllocation(103, 305, 3, datetime(2024, 7, 2)))

```

```

Lab-ASS-12.5.py > ...
315 st cases
316 __name__ == "__main__":
317     system = HostelSystem()
318     # Adding allocation records
319     system.add_allocation(RoomAllocation(101, 202, 2, datetime(2024, 7, 1)))
320     system.add_allocation(RoomAllocation(102, 101, 1, datetime(2024, 7, 3)))
321     system.add_allocation(RoomAllocation(103, 305, 3, datetime(2024, 7, 2)))
322     # Search by student ID
323     student = system.search_by_student_id(102)
324     if student:
325         print(f"Found student: ID {student.student_id}, Room {student.room_number}, Floor {student.floor}, Allocation date {student.allocation_date}")
326     else:
327         print("Student not found.")
328     # Sort by room number
329     system.sort_by_room_number()
330     print("\nAllocations sorted by room number:")
331     for allocation in system.allocations.values():
332         print(f"Student ID {allocation.student_id}, Room {allocation.room_number}, Floor {allocation.floor}, Allocation date {allocation.allocation_date}")
333     # Sort by allocation date
334     system.sort_by_allocation_date()
335     print("\nAllocations sorted by allocation date:")
336     for allocation in system.allocations.values():
337         print(f"Student ID {allocation.student_id}, Room {allocation.room_number}, Floor {allocation.floor}, Allocation date {allocation.allocation_date}")
338

```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Student ID 103, Room 305, Floor 3, Allocation Date 2024-07-02
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>
Student ID 103, Room 305, Floor 3, Allocation Date 2024-07-02
Student ID 103, Room 305, Floor 3, Allocation Date 2024-07-02
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>

Student ID 103, Room 305, Floor 3, Allocation Date 2024-07-02
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>
Student ID 103, Room 305, Floor 3, Allocation Date 2024-07-02
Student ID 103, Room 305, Floor 3, Allocation Date 2024-07-02
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>

Student ID 103, Room 305, Floor 3, Allocation Date 2024-07-02
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>
Student ID 103, Room 305, Floor 3, Allocation Date 2024-07-02
Student ID 103, Room 305, Floor 3, Allocation Date 2024-07-02
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>

Student ID 103, Room 305, Floor 3, Allocation Date 2024-07-02
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>
Student ID 103, Room 305, Floor 3, Allocation Date 2024-07-02
Student ID 103, Room 305, Floor 3, Allocation Date 2024-07-02
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> 
```

Task Description #6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.
2. Sort movies based on rating or release year.

Student Task

- Recommend searching and sorting algorithms using AI.
- Justify the chosen algorithms.
- Implement Python functions.

```

Lab-ASS-12.5.py > Movie > _init_
341 #Task-6: Online Movie Streaming Platform
342 #Prompt:A streaming service maintains movie records with movie ID, title,genre, rating, and release year.
343 #Search movies by movie ID.
344 #Sort movies based on rating or release year.
345 #Recommend searching and sorting algorithms.Justify the chosen algorithms.Implement Python functions.
346 class Movie:
347     def _init_(self, movie_id, title, genre, rating, release_year):
348         self.movie_id = movie_id
349         self.title = title
350         self.genre = genre
351         self.rating = rating
352         self.release_year = release_year
353 class MoviePlatform:
354     def _init_(self):
355         self.movies = {}
356         # Add movie record (using movie_id as the key)
357     def add_movie(self, movie):
358         self.movies[movie.movie_id] = movie
359         # Search movie by movie ID (Optimized search using Hashing)
360     def search_by_movie_id(self, movie_id):
361         return self.movies.get(movie_id, None)
362         # Merge Sort for sorting movies by rating or release year
363     def merge_sort(self, movies, key):
364         if len(movies) > 1:
365             mid = len(movies) // 2
366             left_half = movies[:mid]
367             right_half = movies[mid:]
368             # Recursively split the list
369             self.merge_sort(left_half, key)
370             self.merge_sort(right_half, key)

```

```

Lab-ASS-12.5.py > Movie > _init_
353 class MoviePlatform:
363     def merge_sort(self, movies, key):
371         i = j = k = 0
372         # Merging the lists back together
373         while i < len(left_half) and j < len(right_half):
374             if getattr(left_half[i], key) > getattr(right_half[j], key): # Descending order for rating
375                 movies[k] = left_half[i]
376                 i += 1
377             else:
378                 movies[k] = right_half[j]
379                 j += 1
380                 k += 1
381         while i < len(left_half):
382             movies[k] = left_half[i]
383             i += 1
384             k += 1
385         while j < len(right_half):
386             movies[k] = right_half[j]
387             j += 1
388             k += 1
389         # Sort movies by rating (descending order)
390         def sort_by_rating(self):
391             self.merge_sort(list(self.movies.values()), 'rating')
392         # Sort movies by release year (ascending order)
393         def sort_by_release_year(self):
394             self.merge_sort(list(self.movies.values()), 'release_year')
395     # Test cases
396     if __name__ == "__main__":
397         platform = MoviePlatform()
398         # Adding movie records

```

```
Lab-ASS-12.5.py > Movie > _init_
398 # Adding movie records
399 platform.add_movie(Movie(1, "Inception", "Sci-Fi", 8.8, 2010))
400 platform.add_movie(Movie(2, "The Dark Knight", "Action", 9.0, 2008))
401 platform.add_movie(Movie(3, "The Godfather", "Crime", 9.2, 1972))
402 platform.add_movie(Movie(4, "Pulp Fiction", "Crime", 8.9, 1994))
403 platform.add_movie(Movie(5, "Avengers: Endgame", "Action", 8.4, 2019))
404 # Search by movie ID
405 movie = platform.search_by_movie_id(3)
406 if movie:
407     print(f"Found movie: {movie.title}, Genre: {movie.genre}, Rating: {movie.rating}, Release Year: {movie.release_year}")
408 else:
409     print("Movie not found.")
410 # Sort by rating
411 platform.sort_by_rating()
412 print("\nMovies sorted by rating (highest to lowest):")
413 for movie in platform.movies.values():
414     print(f"{movie.title} - Rating: {movie.rating}, Release Year: {movie.release_year}")
415 # Sort by release year
416 platform.sort_by_release_year()
417 print("\nMovies sorted by release year (oldest to newest):")
418 for movie in platform.movies.values():
419     print(f"{movie.title} - Rating: {movie.rating}, Release Year: {movie.release_year}")
420
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - []
Student ID 103, Room 305, Floor 3, Allocation Date 2024-07-02
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>
Student ID 103, Room 305, Floor 3, Allocation Date 2024-07-02
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>
Student ID 103, Room 305, Floor 3, Allocation Date 2024-07-02
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>
Student ID 103, Room 305, Floor 3, Allocation Date 2024-07-02
```

Task Description #7: Smart Agriculture Crop Monitoring System

An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:

1. Search crop details using crop ID.
2. Sort crops based on moisture level or yield estimate.

Student Task

- Use AI-assisted reasoning to select algorithms.
- Justify algorithm suitability.
- Implement searching and sorting in Python.

Lab-ASS-12.5.py > ...

```
423 #Task-7: Smart Agriculture Crop Monitoring System
424 #Prompt:An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, te
425 #Search crop details using crop ID.
426 #Sort crops based on moisture level or yield estimate.
427 #Justify algorithm suitability.Implement searching and sorting in Python.
428 class Crop:
429     def __init__(self, crop_id, crop_name, soil_moisture, temperature, yield_estimate):
430         self.crop_id = crop_id
431         self.crop_name = crop_name
432         self.soil_moisture = soil_moisture
433         self.temperature = temperature
434         self.yield_estimate = yield_estimate
435
436 class AgricultureSystem:
437     def __init__(self):
438         self.crops = {}
439
440     # Add crop record (using crop_id as key)
441     def add_crop(self, crop):
442         self.crops[crop.crop_id] = crop
443
444     # Search crop by crop ID (Optimized search using Hashing)
445     def search_by_crop_id(self, crop_id):
446         return self.crops.get(crop_id, None)
447
448     # Merge Sort for sorting crops by moisture level or yield estimate
449     def merge_sort(self, crops, key):
450         if len(crops) > 1:
451             mid = len(crops) // 2
452             left_half = crops[:mid]
453             right_half = crops[mid:]
454
455             # Recursively split the list
456             self.merge_sort(left_half, key)
457             self.merge_sort(right_half, key)
458
```

```
Release Notes: 1.111.0  Lab-ASS-11.5.py  Lab-ASS-12.5.py 2  data1.txt  data2.txt  library.py
Lab-ASS-12.5.py > ...
436 class AgricultureSystem:
449     def merge_sort(self, crops, key):
461         # Merging the lists back together
462         while i < len(left_half) and j < len(right_half):
463             if getattr(left_half[i], key) > getattr(right_half[j], key): # Descending order for yield
464                 crops[k] = left_half[i]
465                 i += 1
466             else:
467                 crops[k] = right_half[j]
468                 j += 1
469             k += 1
470
471         while i < len(left_half):
472             crops[k] = left_half[i]
473             i += 1
474             k += 1
475
476         while j < len(right_half):
477             crops[k] = right_half[j]
478             j += 1
479             k += 1
480
481         # Sort crops by soil moisture (ascending order)
482         def sort_by_soil_moisture(self):
483             self.merge_sort(list(self.crops.values()), 'soil_moisture')
484
485         # Sort crops by yield estimate (descending order)
486         def sort_by_yield_estimate(self):
487             self.merge_sort(list(self.crops.values()), 'yield_estimate')
488
489 # Test cases
490 if __name__ == "__main__":
491     system = AgricultureSystem()
492
493     # Adding crop records
494     system.add_crop(Crop(101, "Corn", 30, 25, 500))
```

Ln 513, Col 7 Spaces: 4 UTF-8 CRLF { } Python Python 3.8 (64-bit)

```
Lab-ASS-12.5.py > ...
495     system.add_crop(Crop(102, "Wheat", 40, 22, 700))
496     system.add_crop(Crop(103, "Rice", 35, 28, 650))
497     system.add_crop(Crop(104, "Soybean", 50, 20, 800))
498     system.add_crop(Crop(105, "Barley", 20, 30, 600))
499
500     # Search by crop ID
501     crop = system.search_by_crop_id(102)
502     if crop:
503         print(f"Found crop: {crop.crop_name}, Moisture: {crop.soil_moisture}%, Temperature: {crop.temperature}°C")
504     else:
505         print("Crop not found.")
506
507     # Sort crops by soil moisture
508     system.sort_by_soil_moisture()
509     print("\nCrops sorted by soil moisture (lowest to highest):")
510     for crop in system.crops.values():
511         print(f"{crop.crop_name} - Moisture: {crop.soil_moisture}%, Yield Estimate: {crop.yield_estimate}kg")
512
513     # Sort crops by yield estimate
514     system.sort_by_yield_estimate()
515     print("\nCrops sorted by yield estimate (highest to lowest):")
516     for crop in system.crops.values():
517         print(f"{crop.crop_name} - Moisture: {crop.soil_moisture}%, Yield Estimate: {crop.yield_estimate}kg")
518
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [ ] [X]
C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & C:\Program Files\Python38\python.exe c:/users/spurthi
Student ID 103, Room 305, Floor 3, Allocation Date 2024-07-02

Allocations sorted by allocation date:
Student ID 101, Room 202, Floor 2, Allocation Date 2024-07-01
Student ID 102, Room 101, Floor 1, Allocation Date 2024-07-03
Student ID 103, Room 305, Floor 3, Allocation Date 2024-07-02
Traceback (most recent call last):
  File "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-ASS-12.5.py", line 396, in <module>
    if name == "main":
```

Task Description #8: Airport Flight Management System

An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. The system must:

1. Search flight details using flight ID.
2. Sort flights based on departure time or arrival time.

Student Task

- Use AI to recommend algorithms.
- Justify the algorithm selection.
- Implement searching and sorting logic in Python.

Lab-ASS-12.5.py > ...

```
521 #Task-8: Airport Flight Management System
522 #Prompt:An airport system stores flight information including flight ID,airline name, departure time, arrival time.
523 #Search flight details using flight ID.
524 #Sort flights based on departure time or arrival time.
525 #Justify the algorithm selection.Implement searching and sorting logic in Python.
526 class Flight:
527     def __init__(self, flight_id, airline_name, departure_time, arrival_time, status):
528         self.flight_id = flight_id
529         self.airline_name = airline_name
530         self.departure_time = departure_time
531         self.arrival_time = arrival_time
532         self.status = status
533
534 class AirportSystem:
535     def __init__(self):
536         self.flights = {}
537
538     # Add flight record (using flight_id as key)
539     def add_flight(self, flight):
540         self.flights[flight.flight_id] = flight
541
542     # Search flight by flight ID (Optimized search using Hashing)
543     def search_by_flight_id(self, flight_id):
544         return self.flights.get(flight_id, None)
545
546     # Merge Sort for sorting flights by departure time or arrival time
547     def merge_sort(self, flights, key):
548         if len(flights) > 1:
549             mid = len(flights) // 2
550             left_half = flights[:mid]
551             right_half = flights[mid:]
552
553             # Recursively split the list
554             self.merge_sort(left_half, key)
555             self.merge_sort(right_half, key)
556
```

```

Lab-ASS-12.5.py > ...
534 class AirportSystem:
547     def merge_sort(self, flights, key):
549         while i < len(left_half) and j < len(right_half):
550             if getattr(left_half[i], key) < getattr(right_half[j], key): # Ascending order for both d
551                 flights[k] = left_half[i]
552                 i += 1
553             else:
554                 flights[k] = right_half[j]
555                 j += 1
556             k += 1
557
558         while i < len(left_half):
559             flights[k] = left_half[i]
560             i += 1
561             k += 1
562
563         while j < len(right_half):
564             flights[k] = right_half[j]
565             j += 1
566             k += 1
567
568     # Sort flights by departure time
569     def sort_by_departure_time(self):
570         self.merge_sort(list(self.flights.values()), 'departure_time')
571
572     # Sort flights by arrival time
573     def sort_by_arrival_time(self):
574         self.merge_sort(list(self.flights.values()), 'arrival_time')
575
576 # Test cases
577 if __name__ == "__main__":
578     system = AirportSystem()
579
580
581

```

```
Release Notes: 1.111.0  Lab-ASS-11.5.py  Lab-ASS-12.5.py 3 X  data1.txt  data2.txt  library.py

Lab-ASS-12.5.py > ...
593     system.add_flight(Flight(102, "Airline B", datetime(2024, 7, 1, 14, 00), datetime(2024, 7, 1, 16, 00)),
594     system.add_flight(Flight(103, "Airline C", datetime(2024, 7, 1, 9, 00), datetime(2024, 7, 1, 11, 00)),
595     system.add_flight(Flight(104, "Airline D", datetime(2024, 7, 1, 16, 30), datetime(2024, 7, 1, 18, 30)),
596     system.add_flight(Flight(105, "Airline E", datetime(2024, 7, 1, 11, 30), datetime(2024, 7, 1, 13, 30)),
597
598     # Search by flight ID
599     flight = system.search_by_flight_id(102)
600     if flight:
601         print(f"Found flight: {flight.airline_name} {flight.flight_id}, Departure: {flight.departure_time}")
602     else:
603         print("Flight not found.")
604
605     # Sort flights by departure time
606     system.sort_by_departure_time()
607     print("\nFlights sorted by departure time:")
608     for flight in system.flights.values():
609         print(f"{flight.airline_name} {flight.flight_id}, Departure: {flight.departure_time.strftime('%Y-%m-%d %H:%M')}")
610
611     # Sort flights by arrival time
612     system.sort_by_arrival_time()
613     print("\nFlights sorted by arrival time:")
614     for flight in system.flights.values():
615         print(f"{flight.airline_name} {flight.flight_id}, Departure: {flight.departure_time.strftime('%Y-%m-%d %H:%M')}")

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + v [ ] [ ] [ ]
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & C:\Program Files\Python38\python.exe C:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-ASS-12.5.py
Student ID 101, Room 202, Floor 2, Allocation Date 2024-07-01
Student ID 102, Room 101, Floor 1, Allocation Date 2024-07-03
Student ID 103, Room 305, Floor 3, Allocation Date 2024-07-02

Allocations sorted by allocation date:
Student ID 101, Room 202, Floor 2, Allocation Date 2024-07-01
Student ID 102, Room 101, Floor 1, Allocation Date 2024-07-03
Student ID 103, Room 305, Floor 3, Allocation Date 2024-07-02
Traceback (most recent call last):
  File "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-ASS-12.5.py", line 396, in <module>
    if __name__ == "__main__":
```